Computers and Chemical Engineering 156 (2022) 107531

Contents lists available at ScienceDirect

Computers and Chemical Engineering

journal homepage: www.elsevier.com/locate/compchemeng

Fault detection and diagnosis for non-linear processes empowered by dynamic neural networks

Georgios Gravanis[a,*], Ioannis Dragogias[a], Konstantinos Papakiriakos[a], Chrysovalantou Ziogou[b], Konstantinos Diamantaras[a]

[a] International Hellenic University, Greece
[b] Centre for Research and Technology Hellas, Chemical Process & Energy Resources Institute, Greece

ARTICLE INFO

Article history:
Received 1 June 2021
Revised 5 September 2021
Accepted 7 September 2021
Available online 30 September 2021

Keywords:
Recurrent neural networks
Dynamic neural networks
Fault detection
Chemical processes

ABSTRACT

In the era of the 4th industrial revolution, a key challenge for the industries is the efficient reduction of the production cost caused by malfunctioning equipment. This paper proposes a Fault Detection and Diagnosis (FDD) framework for Non-Linear Processes utilizing Dynamic Neural Networks and feature reduction methods. We investigate both types of dynamic neural models, ie. Recurrent Neural Networks -in particular Long Short-Term Memory (LSTM) models, and Time Delay Neural Networks (TDNN). Intending to mitigate the overfitting problem, we also investigated the use of feature reduction techniques such as Non-Negative Matrix Factorization (NMF), Principal Component Analysis (PCA), and kernel PCA (kPCA), as preprocessing steps in our Machine Learning pipeline. The Tennessee Eastman Process (TEP) is used to evaluate our proposed framework on 18 different faults. Our simulations demonstrate that our method outperforms state of the art methods in the majority of those faults.

© 2021 Elsevier Ltd. All rights reserved.

1. Introduction

Industry 4.0, which is regarded as the fourth industrial revolution, targets transforming factories into smart factories. The purpose of smart factories is to address and overcome the challenges of stiff competition, shorter production requirements, and highly customized products. For the chemical process industry, smart factories are expected to maximize the economic value of processes and to optimize the management of their resources, while applying the optimum management of processes to avoid safety incidents (Christofides et al., 2007). Since chemical processes are of dynamic nature, their safe operation under dynamically changing conditions and their sustainability should be the basic metrics of smart factories (Sansana et al., 2021). Although several simple logic control systems, such as Distributed Control Systems (DCS), Supervisory Control And Data Acquisition systems (SCADA), and Programmable Logic Controllers (PLC), have been widely employed in chemical processes, abnormal event management still faces many challenges. The efficient handling of abnormal events intends to detect and diagnose situations as soon as possible, and provide timely and reliable decision support to operators to apply proper actions to prevent the escalation of disturbances (Venkatasubramanian et al., 2003).

Nowadays, the industry has started to embrace newer technologies concerning instrumentation and monitoring of production processes. With the emergence of Industry 4.0 (Qin et al., 2016), industrial processes are now presented with unique opportunities in terms of key enabling technologies, such as Industrial Internet of Things (IIoT), Cloud Computing, and Deep Learning (DL) (Pistikopoulos et al., 2021). The deployment of these technologies target to optimize their performance, achieve desirable operation and process control (Iqbal et al., 2019).

Amongst others, a significant aspect that is tackled by Industry 4.0 is the amount of data generated and logged by several devices that contribute to the monitoring of industrial processes. Such data can be transformed into actionable knowledge related to the operation of production lines. An example of that knowledge is Fault Detection since, knowing if any fault has occurred during production as early as possible, can lead to decisions that will increase productivity and reduce operation downtime (Angelopoulos et al., 2020).

Overall the development of Fault Detection and Diagnosis (FDD) systems for complex nonlinear industrial processes is difficult and poses many challenges. This work proposes an FDD framework, with implementation considerations for chemical processes of any scale. This FDD framework is developed for nonlinear dynamic pro-

* Corresponding author.
E-mail addresses: ggravanis@iee.ihu.gr (G. Gravanis), dragogias@ihu.gr (I. Dragogias), it103604@it.teithe.gr (K. Papakiriakos), cziogou@certh.gr (C. Ziogou), kdiamant@ihu.gr (K. Diamantaras).

https://doi.org/10.1016/j.compchemeng.2021.107531
0098-1354/© 2021 Elsevier Ltd. All rights reserved.

cesses and relies on dynamic neural networks. As a benchmark process system for the evaluation of the proposed framework, we use the Tennessee Eastman Process (TEP) (Downs and Vogel, 1993).

Within the proposed FDD framework, a systematic study combining several parameters i.e. sampling time, feature reduction techniques, and state of the art algorithms for time series classification is performed.This study resulted in a comprehensive analysis that highlights the importance of each of the aforementioned parameters in the fault detection task.Moreover, the proposed framework outperforms similar studies using Deep Learning algorithms for fault detection in TEP. In summary, the main contributions of this work are the following:

- **Produce a powerful Fault Detection and Diagnosis framework**, utilizing state of the art dynamic Deep Learning models and techniques for time series analysis such as *Long Short Term Memory* (LSTM) and *Time Delay Neural Networks* (TDNN).The proposed FDD framework produces better results compared to similar frameworks.
- **Evaluate the usefulness of common feature reduction methods**, *Principal Component Analysis* (PCA) and *Non-negative Matrix Factorization* (NMF) for the fault detection in chemical processes. Except for PCA and NMF, PCA with rbf kernel (kPCA) is utilized to capture non-linearity in feature reduction preprocessing step.
- **Identify the importance of understanding the operation of each process** before creating a generic Deep Learning pipeline for fault detection. More specific, it is explained why TEP Faults 15 and 16 should be excluded from the classifier.

The paper is organized as follows: Section 2 presents related work about fault detection and identification in TEP. Then, Section 3 describes the fundamentals of TE process along with its complexity issues. Next, Sections 4 and 5 briefly describe the feature reduction and the Deep Learning algorithms used in this study, whereas in Section 6 we present the experimental procedure and the evaluation of the results. Finally, in Section 7 we summarize the conclusions of this work.

2. Related work

As described in Section 1, optimization of chemical processes can be performed by various approaches, one of them is through robust and efficient early fault detection and diagnosis, which is a problem that concerns the chemical industry from its very beginning. Depending on the technological solutions and tools of each period, different approaches for fault detection have been proposed by researchers and engineers during the last decades. Since the TE process was introduced in 1993, numerous studies with several approaches have been conducted until now. In general, we can separate the related work into those that utilize solutions such as Statistical Process Control, Machine Learning and Deep Learning algorithms.

One of the first attempts was from Lin et al. (2000) who used a nonlinear dynamic PCA with feed-forward neural network. Akbaryan and Bishnoi (2001) used Decision Trees, Kano et al. (2002) used moving principal component analysis, Chen and Liao (2002) used shallow Neural Networks with PCA, Maurya et al. (2005) used a PCA - Qualitative trend analysis, while (Chiang et al., 2004) and (Zhang, 2009) used Support Vector Machines (SVMs) and some alterations of them for fault classification. Also, a different approach from (Odiowei and Cao, 2010) with a novel state-space independent component analysis, Eslamloueyan (2011) used Hierarchical neural network,(Lau et al., 2013) used multi-scale PCA with adaptive neuro-fuzzy inference system, Rad and Yazdanpanah (2015) used multilayer perceptron based on Expectation-Maximization (EM) clustering.

However, most of the above-mentioned studies include only a part of TEP faults and use either statistical process control or Machine Learning algorithms with inferior results when compared to studies using Deep Learning methods.

Therefore, the most recent and relevant studies to our approach that are utilizing Deep Learning algorithms will be presented next.

More specifically, Zhang and Zhao (2017) used a Deep Belief Network and faced fault detection in TEP as a multiclass classification problem. The authors created a classifier including all available faults, including faults 15 and 16, in which the classification accuracy is poor, while they recognize that this phenomenon should be further investigated. As a data preprocessing step, they perform feature selection by applying Mutual Information. The sampling rate of the process is 3 min as introduced by Downs and Vogel (1993). The average classification performance on 20 faults, without including the normal operation is 82.1%.

Wu and Zhao (2018) used a Deep Convolutional Neural Network (DCNN). The data are transformed into matrices of $n * m$ dimensions where n is the number of the variables and m is the time window used. The authors present results of two classifiers, one trained and tested in all 20 faults, and a second one where faults 9, 15 and 16 are excluded without providing any further explanation. The overall performance for the first classifier which includes all faults is 88.2%. However, the authors choose not to use either a feature reduction or feature selection method. To our opinion, in case of implementing the proposed framework in a larger system with more measured variables, a feature reduction method should be implemented as a preprocessing step due to possible existence of correlated features.

Zhang et al. (2019) propose the use of two Neural Networks sequentially, an RNN for feature reduction and a Bidirectional LSTM (BiLSTM) as a classifier.Similar to previously mentioned studies, they include both faults 15 and 16 in their classifier, while they recognize that the proposed method scores relatively small accuracy in faults 15 and 16 compared to other faults. The authors use a sampling rate of 2 min (30 samples/h) which is faster than other studies. The overall accuracy achieved is 92.7% for all faults including normal operation.

Another approach is that of Xie et al. (2019); Zhao et al. (2019), which utilize a classifier for each fault separately. Taking that approach into account, related work can be separated in two main categories: ***i.****those facing the problem as a fault vs. normal classification problem* and ***ii.****those considering fault detection as a multiclass problem. Since in this study fault detection is considered as a* multiclass problem*, studies considering the problem as fault vs. normal classification are not further presented.*

Despite the aforementioned advancements, data-driven fault detection and diagnosis techniques have still room for improvement. Moreover, the impact of the proper feature reduction method is explored. More specifically, *kPCA, PCA, NMF* (Lee and Seung, 1999), *Mutual Information, (MI)* and *minimum Redundancy Maximum Relevance, (mRMR)* (Peng et al., 2005) are applied to get the optimum results for the fault detection task. Concerning the classification algorithms, two different neural networks architectures for sequence classification i.e. *Time Delay Neural Networks* and *Long Short Term Memory Neural Networks*, are utilized.

3. Description of Tennessee Eastman process

The Tennessee Eastman process was designed by the Eastman Chemical Company (Downs and Vogel, 1993) and it has been used as a reference benchmark case for numerous studies, such as process monitoring, process control, fault detection, and diagnosis. The process consists of five primary units i.e. a Reactor, a Condenser, a Stripper, a Separator, and a Compressor, where chemicals G and H are produced from feedstocks A, C, D, and E with byproduct F, and

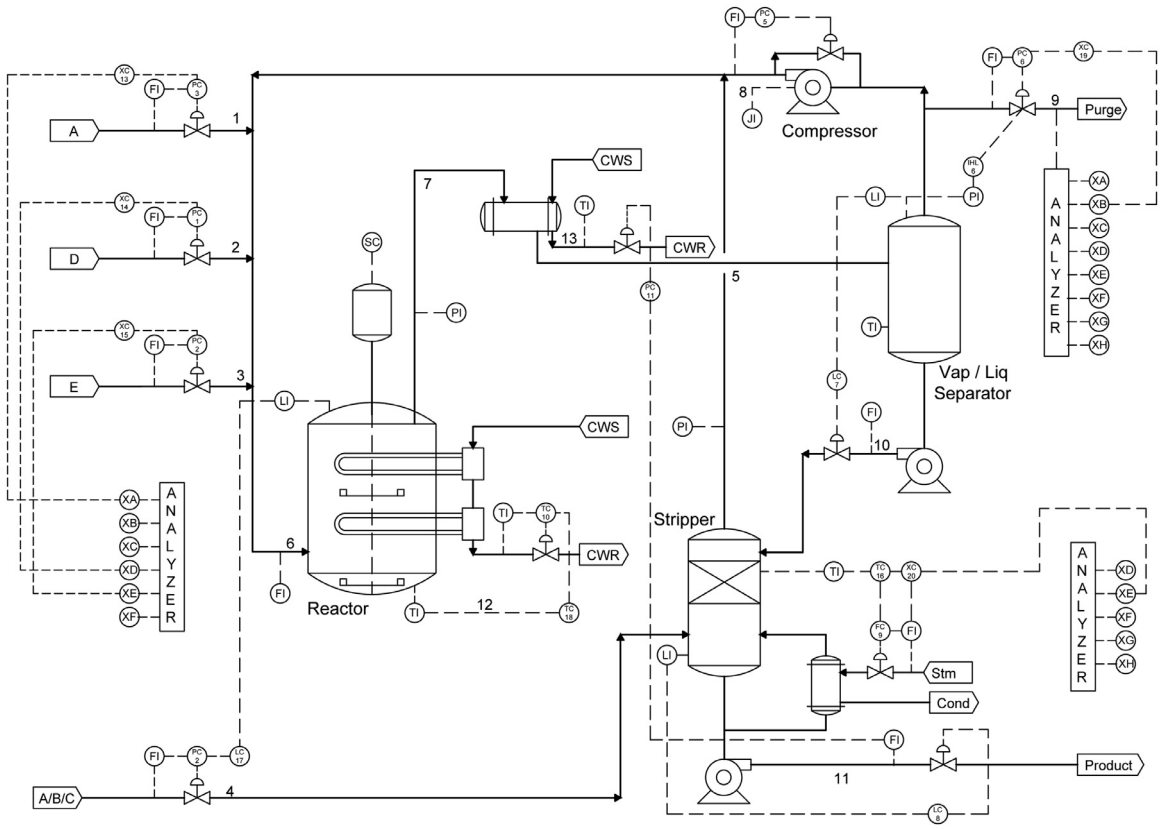

Fig. 1. TEP Piping and Instrumentation Diagram as proposed by Downs and Vogel.

inert compound B. These units are the ones that are the most frequently used in chemical processes. Therefore, the proposed FDD framework can be adapted to the respective unit of other chemical processes.

Next, a short description of the Piping and Instrumentation Diagram (P&ID), along with the predefined disturbances and the parameters used for the data generated for this study will be presented.

3.1. Piping and instrumentation diagram (P&ID)

In general, a P&ID (Fig. 1) describes the operational flow and the instrumentation used for monitoring and controlling a process. Each node depicts the type of instrumentation used to monitor the operating conditions of the unit and acquires real-time measurements. TE process has 41 nodes that belong to process measurements. From this point forward and for the rest of the paper we will refer to those measurements as *xmeas* (Table 1). Besides the xmeas, there are twelve more variables that refer to manipulated values *xmvs* (Table 2). The values of the manipulated variables are dictated by the process control strategy. In this work, both *xmeas* and *xmvs* are used as features for the training and the testing of the fault detection models.

3.2. Analysis of TEP disturbances

Initially, the TE process was used as a benchmark problem for the optimization of the process control in large-scale chemical processes (Downs and Vogel, 1993). As such, the authors introduced 20 predefined disturbances, and control engineers had to tackle them by proper control tuning. In the first edition of the TE process, the authors kept disturbances 16–20 “Unknown” for the engineers. However, in the latest release of the TEP by Bathelt et al. (2015), those disturbances are explicitly described. All predefined disturbances along with their description are presented in Table 3.

In general, the predefined disturbances of TEP, belong to the basic categories that a chemical process could face *i.e. step changes in a process variable, variability increment, time drifting, and valve malfunctions*, while the impact of each disturbance for the process, depends on the control strategy and the operational mode used.

This study compares how classification algorithms perform under different operational modes and optimized control strategies as defined by Ricker (1996) and Larsson et al. (2001). Moreover, we investigate the importance of using proper feature reduction methods and the size of the time window for recognizing whether a disturbance has occurred or not.

3.3. Generation of datasets

In the first edition of the TE process introduced by Downs and Vogel (1993), the simulation code was written in *FORTRAN*, while the latest revision of the TE process which is available online is introduced by Bathelt et al. (2015). In that version the authors rewrote the simulation code for *MATLAB* environment with two production modes. This updated model includes the implementation of two control strategies. Mode 1 and Mode 3 implement the decentralized control strategy by Ricker (1996), without and with the override loops, respectively. The third mode relies on Mode 1 and implements a self-optimizing control strategy presented by Larsson et al. (2001). Moreover, the authors optimized the code for faster simulation and added some further features such as monitoring of the outputs of the process disturbances and some more process measurements in comparison with the original model. But most importantly the updated model tackled an issue where the simulation results showed a dependency on the chosen

Table 1
Process and analyzers measurement variables.

	Process measurements		Composition measurements		
Variable	Description	Units	Variable	Description	Stream
XMEAS(1)	A Feed (Stream 1)	kscmh	XMEAS(23)	Component A	6
XMEAS(2)	D Feed (Stream 2)	kg/h	XMEAS(24)	Component B	6
XMEAS(3)	E Feed (Stream 3)	kg/h	XMEAS(25)	Component C	6
XMEAS(4)	Total Feed (Stream 4)	kscmh	XMEAS(26)	Component D	6
XMEAS(5)	Recycle Flow(Stream 8)	kscmh	XMEAS(27)	Component E	6
XMEAS(6)	Reactor Feed Rate (Stream 6)	kscmh	XMEAS(28)	Component F	6
XMEAS(7)	Reactor Pressure	kPa gauge	XMEAS(29)	Component A	9
XMEAS(8)	Reactor Level	%	XMEAS(30)	Component B	9
XMEAS(9)	Reactor Temperature	Deg C	XMEAS(31)	Component C	9
XMEAS(10)	Purge Rate (Stream 9)	kscmh	XMEAS(32)	Component D	9
XMEAS(11)	Product Sep Temp	Deg C	XMEAS(33)	Component E	9
XMEAS(12)	Product Sep Level	%	XMEAS(34)	Component F	9
XMEAS(13)	Product Sep Pressure	kPa gauge	XMEAS(35)	Component G	9
XMEAS(14)	Product Sep Underflow (Stream 10)	m^3/h	XMEAS(36)	Component H	9
XMEAS(15)	Stripper Level	%	XMEAS(37)	Component D	11
XMEAS(16)	Stripper Pressure	kPa gauge	XMEAS(38)	Component E	11
XMEAS(17)	Stripper Underflow (Stream 11)	m^3/h	XMEAS(39)	Component F	11
XMEAS(18)	Stripper Temperature	Deg C	XMEAS(40)	Component G	11
XMEAS(19)	Stripper Steam Flow	kg/h	XMEAS(41)	Component H	11
XMEAS(20)	Compressor Work	kW			
XMEAS(21)	Reactor Cooling Water Outlet Temp	Deg C			
XMEAS(22)	Separator Cooling Water Outlet Temp	Deg C			

[1] Units of Composition measurements are mole%.

Table 2
Manipulated values.

Variable	Description	Units
XMV(1)	D Feed (Stream 2)	kg/h
XMV(2)	E Feed (Stream 3)	kg/h
XMV(3)	A Feed (Stream 1)	kscmh
XMV(4)	Total Feed (Stream 4)	kscmh
XMV(5)	Compresor Recycle Valve	%
XMV(6)	Purge Valve (Stream 9)	kscmh
XMV(7)	Separator Pot Liquid Flow (Stream 10)	m^3/h
XMV(8)	Stripper Liquid Product Flow (Stream 11)	m^3/h
XMV(9)	Stripper Steam Valve	Deg C
XMV(10)	Reactor Cooling Water Flow	kscmh
XMV(11)	Condenser Cooling Water Flow	Deg C
XMV(12)	Agitator Speed	%

solver and simulation time-increment, in cases where disturbances of the type random variation are activated.

The data generated for this study includes the three aforementioned operational modes and control strategies. For each case, the same parameters concerning disturbance introduction (10th hour of operation), total operation time (50 h), and sampling rate (36 s) are used. The sampling rate is defined at 50 samples/h (1 sample / 36 s) as suggested by the newer version of the TEP code provided by Bathelt et al. (2015). In addition, for each fault 50 simulations are performed with different random seed generator along with the normal fault-free operation case. Therefore, for each version, we have a total of 250,000 samples per fault, except for fault 6 which leads to unexpected unit shut down and the total samples are 85,750. However, since no significant variations were observed during experimentation between different operational modes, the results presented in this study are about operational Mode 1 which is the one used in all similar studies.

Table 3
Predefined disturbances in TEP.

Variable	Description	Type
IDV(1)	A/C feed ratio, B composition constant (Stream 4)	step
IDV(2)	B composition, A/C ratio constant (Stream 4)	step
IDV(3)	D feed temperature (Stream 2)	step
IDV(4)	reactor cooling water inlet temperature	step
IDV(5)	condenser cooling water inlet temperature	step
IDV(6)	A feed loss (Stream 1)	step
IDV(7)	C header pressure loss - Reduced availability (Stream 4)	step
IDV(8)	A, B, C, feed composition (Stream 4)	random
IDV(9)	D feed temperature (Stream 2)	random
IDV(10)	C feed temperature (Stream 4)	random
IDV(11)	Reactor cooling water inlet temperature	random
IDV(12)	Condenser cooling water inlet temperature	random
IDV(13)	Reaction kinetics	slow drift
IDV(14)	Reactor cooling water valve	sticking
IDV(15)	Condenser cooling water valve	sticking
IDV(16)	(unknown) Deviations of heat transfer within stripper (heat exchanger)	random
IDV(17)	(unknown) Deviations of heat transfer within reactor	random
IDV(18)	(unknown) Deviations of heat transfer within condenser	random
IDV(19)	(unknown) re-cycle valve of compressor, underflow separator (stream 10), underflow stripper (stream 11) and steam valve stripper	sticking
IDV(20)	Unknown	random

The revised code can be found here,[1] while the generated dataset can be available upon request. From this point and on the disturbances will be referred to as Faults.

3.4. Excluding fault 15 and fault 16

Before proceeding with the next steps of our work, it is crucial to refer to Faults 15 and 16 and their physical interpretation for the TE process.

As mentioned before, for this study we follow the control strategies as proposed by Ricker (1996) and Larsson et al. (2001). In both strategies it is explicitly mentioned that the valve that controls the steam inlet of the heat exchanger is closed. Therefore, by looking at the P&ID diagram and by keeping in mind that the stripper steam inlet control valve is closed, any variation in the temperature of that steam stream does not affect the process at all. As such, for the rest of this work Fault 16 is excluded.

Concerning Fault 15, which corresponds to a problem at the valve of the condenser's cooling water line, several studies (Russell et al., 2000), Yin et al. (2012), Ghosh et al. (2014), Rad and Yazdanpanah (2015), mention that it belongs to a group of faults (3, 9 and 15) which have almost no influence on product quality. Moreover, according to that studies that group of faults can be hardly detected by statistical data-driven detection methods. On the contrary, Neural Network based classifiers have been proven to easily detect Faults 3 and 9. However, Fault 15 still remains hard to detect by any model, mostly because of its similarity with the Normal state operation as explained in Zhang and Zhao (2017).

For this study, and since the operation with Fault 15 is very similar to the Normal state as explained before, we choose to exclude it from our classifier.

4. Feature reduction

In production processes several variables are monitored and logged to ensure the safe operation and the product quality. From the perspective of Deep Learning, those measured variables along with the manipulated variables produced by the process controllers, can be used as features for the training and the testing procedure of classification algorithms. However, not all of those variables provide information about the condition of the process, while several of them may be correlated. In this study, *Non-Negative Matrix Factorization* and *Principal Component Analysis* are utilized for performing feature reduction and dimensionality reduction. Moreover, *minimum Redundancy Maximum Relevance* and *Mutual Information* are tested with the overall accuracy to be 3%–7% lower than kPCA, PCA and NMF methods. Therefore, mRMR and MI results are not presented in this study.

4.1. Non-negative matrix factorization (NMF)

Non-Negative Matrix Factorization is a relatively new method for dimensionality reduction. The key for implementing NMF is that data must not contain negative values, which is the main idea behind algorithm's definition as proposed by Lee and Seung (1999). Thus, NMF is widely used in Nature Language Processing, Image Recognition and Biomedical applications. As far as we know, there is no prior work implementing NMF for feature reduction in chemical processes. Moreover, the majority of the measured variables in chemical processes, and more specifically in our case the TEP as described in Section 3, have positive values. However, during data generation some negative values were detected. A careful examination showed that there are some measurements with values slightly below zero as depicted in Fig. 2. Since those values are not significant for the training and the testing of the DL models, and for the implementation of NMF method in this study, those values are considered equal to zero.

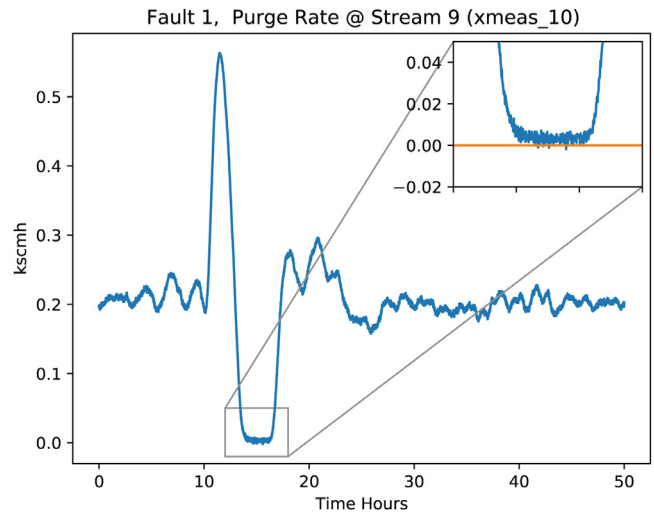

Fig. 2. Purge Rate @ Stream 9 response when disturbance 1 is activated.

4.2. Principal components analysis (PCA)

PCA is a statistical method for linear dimensionality reduction. It performs dimensionality reduction by integrating the data into a linear subsystem of lower dimension. This method constructs a lower-dimensional representation of the data with the maximum amount of variation. PCA describes most of the variance of the data values in lower-dimension and that removes the correlation of the values. For this study the standard scikit learn PCA method is used.

4.3. Kernel principal components analysis (kPCA)

Since the chemical process under study is non-linear, it is reasonable to examine non-linear feature reduction methods, such as *Kernel PCA* (Scholkopf et al., 1999). This is an enhanced version of PCA which employs the "kernel trick" to map the data into high dimensional space. For our study, we employ the popular Gaussian *(RBF)* kernel, which has very powerful representation capabilities.

The main drawback of using kernel PCA is the calculation of the $m * m$ kernel matrix K, where m is the number of observations. In this study, m refers to the number of samples taken through time. As explained in Section 3.3, the total number of samples per fault is 250,000 except fault 6 where the number of samples is 85,750. Those numbers, combined with the multiclass approach followed by this study, produces a very large matrix that has to be calculated. To overcome this problem, we sub-sampled the data by a factor s. Thus the size of matrix K, was reduced s times in both i and g dimensions, only for the training phase of kPCA. The best value of the hyper-parameter for TEP was found to be $s = 60$, after an extensive grid search in the range 10–120 with step 10.

5. Algorithm description

As we discussed extensively in Section 2, several works for FDD using TEP have been conducted since the problem was first introduced using SPC, ML and DL methods. In this work we leverage state of the art algorithms such LSTM and TDNN are for fault detection. In this section, a short description of the algorithms used will be presented.

[1] http://depts.washington.edu/control/LARRY/TE/download.html

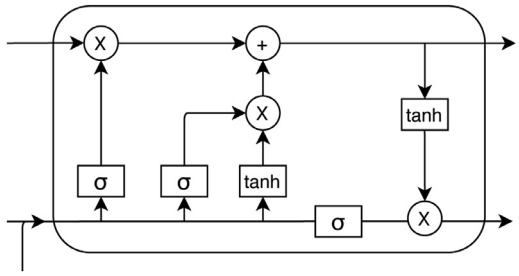

Fig. 3. LSTM cell.

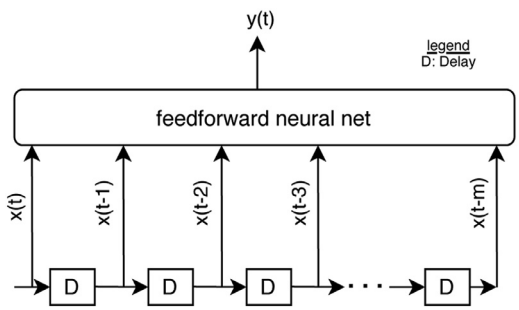

Fig. 4. TDNN architecture.

5.1. Long short-term memory (LSTM)

LSTM as proposed by Hochreiter and Schmidhuber (1997) uses a similar architecture with Recurrent Neural Networks (RNN), and deals with the vanishing gradient problem that RNNs usually face, especially when the last are dealing with large time-series data sets (Fig. 3).

LSTM model, has a vector $c(t)$ which describes the state of the model, and through a time delay, the state vector feedback itself:

$$c(t) = f_i(t)c_i(t-1) + g_i^{in}(t)in_i(t) \tag{1}$$

This model introduced the gate perception and the cell state (memory). The gates can take values between 0 and 1, while the cell state propagates the significant information throughout the processing of the sequence. Therefore, information travels from the first to the last time steps, reducing the effects of short-term memory. As the cell state goes on, information is added or removed to the cell state via gates. The gates of the LSTM cell are described next.

- **Input gate**

$$\mathbf{g}^{in}(t) = [g_1^{in}(t), \ldots, g_n^{in}(t)], 0 \le g_i^{in}(t) \le 1 \tag{2}$$

This gate controls the input to the cell.

$$in_i(t) = \begin{cases} 0, & g_i^{in} = 0 \\ \sigma^{in}(\mathbf{w_i}^{in^T}\mathbf{x}(t) + \mathbf{r_i}^{in^T}\mathbf{y}(t-1) + b^{in}), & g_i^{in} = 1 \end{cases} \tag{3}$$

- **Output gate**

$$\mathbf{g}^{out}(t) = [g_1^{out}(t), \ldots, g_n^{out}(t)], 0 \le g_i^{out}(t) \le 1 \tag{4}$$

This gate controls the cell state $\mathbf{c}(t)$ towards the output.

$$c_i(t) = \begin{cases} 0, & g_i^{out} = 0 \\ \sigma^{out}(c_i(t)), & g_i^{out} = 1 \end{cases} \tag{5}$$

- **Forget gate**

$$f(t) = [f_1(t), \ldots, f_n(t)], 0 \le f_i(t) \le 1 \tag{6}$$

The forget gate $\mathbf{f}$ decides how much of the previous cell state information $\mathbf{c}(t-1)$ is going to be forgotten.

$$c_i(t) = \begin{cases} g_i^{in}(t)in_i(t), & f_i = 0 \\ c_i(t-1) + g_i^{in}(t)in_i(t), & f_i = 1 \end{cases} \tag{7}$$

if $f_i = 1$, the model remembers the previous state element $c_i(t-1)$. If $f_i = 0$, the previous state element $c_i(t-1)$ is immediately forgotten.

5.2. Time delay neural network

Time Delay Neural Network (Fig. 4) is a special version of Recurrent Neural Networks (RNN) in which m previous inputs and the current output y are used:

$$y(t) = f\big(x(t), x(t-1), \ldots, x(t-m)\big) \tag{8}$$

The function f corresponds to a Multilayer Perceptron (MLP) neural network with the input of this network to be the concatenated array of m previous inputs as described in Eq. (9).

$$x(t) = \left[(x(t)^T, x(t-1)^T, \ldots, x(t-m)^T\right]^T \tag{9}$$

The m parameter in our case is the *Time window* parameter as described previously. The data transformation for the TDNN architecture is depicted in Fig. 5 for $Timewindow = 3$.

For this study, the TDNN model is formed out of four dense layers. The activation function used is the rectified linear activation function, *relu*, while the last layer of the classifier is a softmax function with n outputs where n is equal to the number of faults studied plus the normal operation.

Moreover, for the training procedure of both LSTM and TDNN algorithms, techniques such as *reduce learning rate on plateau* and *Early stopping* as they can be found in Keras library[2] are utilized.

The architecture of both LSTM and TDNN models used in this study are depicted in Figs. 6 and 7. Concerning the inputs of models, the data are chunked into sequence matrices of size (n, m) where n is the number of the components and m the time window used. For example, when we refer to $components = 5$ and $timewindow = 5$, matrices of size (5,5) and (25,5) will be the inputs for LSTM and TDNN models respectively.

[2] https://keras.io/

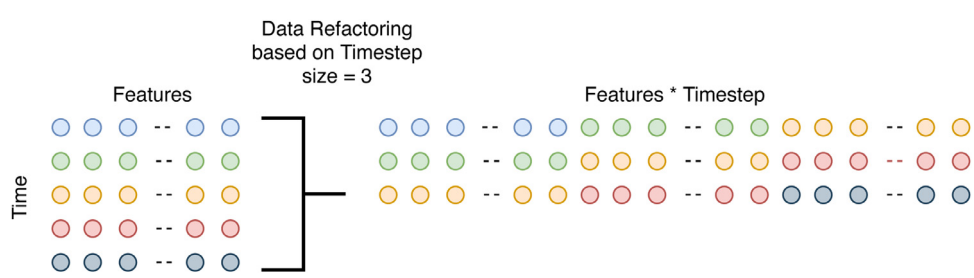

Fig. 5. Data Transformation for TDNN.

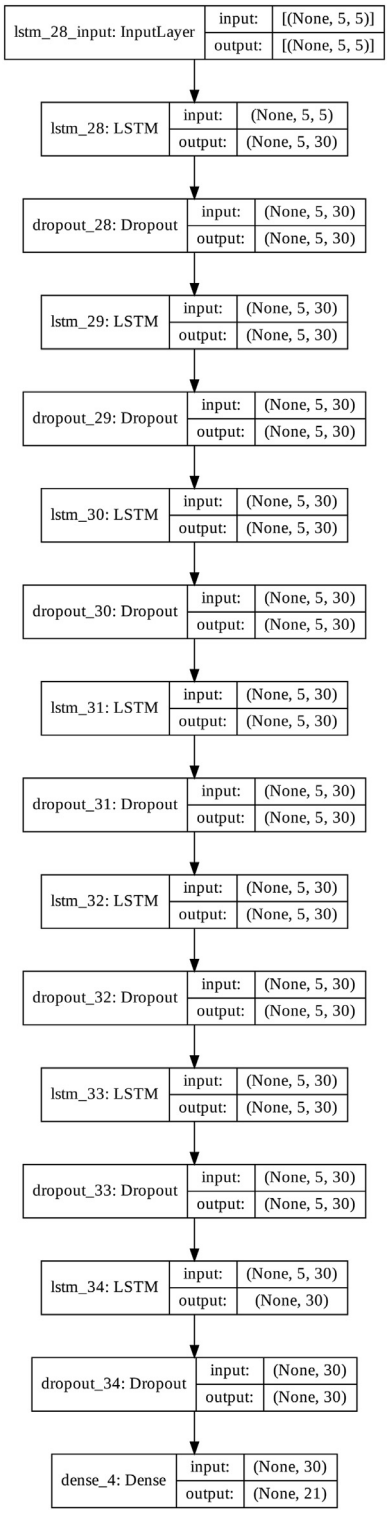

Fig. 6. LSTM model architecture.

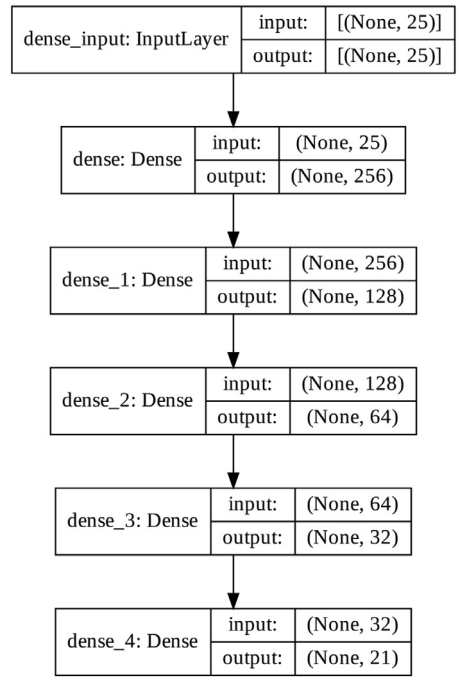

Fig. 7. TDNN model architecture.

5.3. Which metrics are used and why normal operation is considered as a separate class

As discussed previously in Section 2, several studies have been conducted using the TEP benchmarking problem for fault detection. However, there is a wide variance between those studies on the metrics used for the evaluation of their results. Zhang and Zhao (2017); Wu and Zhao (2018); Heo and Lee (2018) use

$$Fault\ Detection\ Rate = \frac{True\ Positive}{True\ Positive + False\ Negative} \tag{10}$$

and

$$Fault\ Prediction\ Rate = \frac{False\ Positive}{False\ Positive + True\ Negative} \tag{11}$$

which should be used together for the evaluation of the results. Apart from Fault Detectin Rate (FDR) and Fault Prediction Rate (FPR), *Accuracy* is also used by Zhao et al. (2018), and Xie et al. (2019). Moreover, there are different approaches on how those studies deal with the normal operation of the system. Some studies use data in which the process operates normally up to a specific point and then a disturbance is triggered, so FDR and FPR metrics are appropriate for the evaluation of their methods. In other studies, the data used for the testing procedure contain only one class each time, either data with a fault introduced or data observed during normal operation. Thus, for each sequence under test we know the actual class in which the sequence belongs to as well as the predicted class obtained from the classifier. Moreover, the classes in the data tested are balanced. In this case, the *Accuracy* criterion can fully describe the efficiency of the classifier. One more advantage of using the accuracy criterion, is that since only one metric is needed, results can be easily inspected. We choose to follow the second approach for the deployment and evaluation of our method.

6. Experimental procedure

To evaluate the aforementioned classification methods for the FDD framework, several experiments are conducted. A standard preprocessing pipeline which includes scaling and feature reduction steps is applied.

Since this study aims not only in high classification results, but in achieving those as quick as possible, time windows of five different lengths are tested to optimize fault detection task.

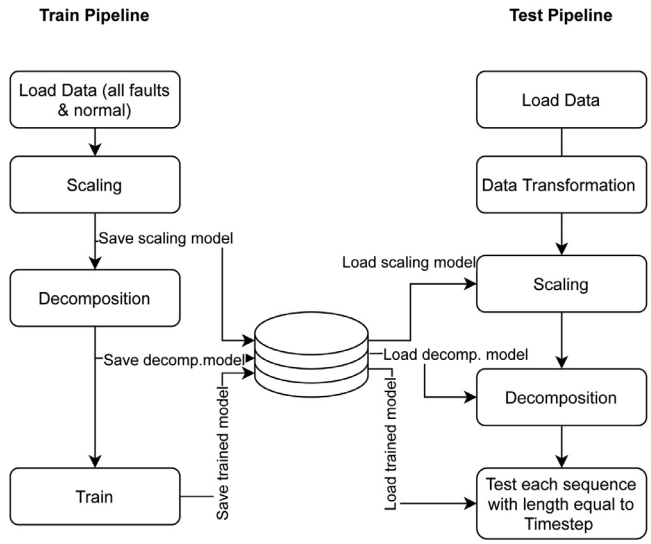

Fig. 8. Training and Testing pipeline.

In summary, the results presented in this section includes experiments with LSTM and TDNN Deep Learning classification algorithms, NMF and both PCA and kPCA feature reduction methods, and time windows between five and twenty-five samples, with step equally to five, which corresponds to 3, 6, 9, 12 and 15 min of operation time respectively.

6.1. Algorithm training and testing pipeline

For the training and the testing procedure of the algorithms a standard pipeline is used. The labeled data are first scaled in range [0–1] and then, a decomposition method is applied to the them. From the preprocessing steps described before the parameters produced during the training phase are saved to be used on the data of the testing procedure. The train set used is the 80% of the generated data (70% for the training and 10% for the validation during the training procedure), while the test set is the rest 20% of the data.

As described before, each model is tested for the normal operation and for all faults, and for all operational modes. The proposed pipeline is depicted in Fig. 8 and Algorithm 1.

Algorithm 1: Sliding window prediction for testing.

t = *Time window size*;
$\mathbf{s}_1, \ldots, \mathbf{s}_T$ = *sample sequence*;
model() = pre-trained model;
for $i = 1$ **to** T **do**
  $W_i = [\mathbf{s}_i : \mathbf{s}_{i+t-1}]$;
  $\text{output}_i = \text{model}(W_i)$
end

6.2. Results

In Fig. 9 the average accuracy for all combinations of time windows and components for both LSTM and TDNN methods and for NMF, PCA, and kPCA decomposition methods is presented. Results of NMF method are poor compared to PCA and kPCA independently of the classification algorithm. Although the features are positive as discussed in Section 4.1 our results indicate that NMF is not the most efficient feature reduction method for Fault Detection in chemical processes. On the contrary, classification results of both LSTM and TDNN when having either PCA or kPCA as a feature reduction preprocessing step, outperform the accuracy of state of the art methods. However, best results are obtained when using LSTM compared to TDNN.

After a more careful examination of the results, we can see that when using less than 30 components, kPCA outperforms PCA although both methods yield their best results for number of components $n > 30$. For $n > 30$ we see that the performance of kPCA and PCA are very close, which indicates that for those many components, PCA can also capture the main non-linearity of the process. Moreover, in Figs. 10 and 11 it is depicted that there is no need to use more than 40 components for Fault Detection in TEP.

Finally, TDNN tends to achieve its best results sooner than LSTM. Since the overall difference in accuracy between two algorithms is about 1%, it may be a matter of model optimization to cover that difference and produce results equally or even better than LSTM. Moreover, it gives the opportunity to the process engineer to choose between a model with predetermined memory (TDNN) and a model with infinite memory as LSTM is. Using a model with predetermined memory may provide an advantage in case of multiple or sequential occurrence of faults in a chemical process, because the error does not propagate in the learning process of the algorithm.

Since most studies use datasets with the default 3 min sampling rate that (Downs and Vogel, 1993) suggested, having a five

		LSTM										TDNN									
		5	10	15	20	25	30	35	40	45	50	5	10	15	20	25	30	35	40	45	50
NMF	5	57.0	73.4	86.4	82.1	85.6	88.8	91.5	94.7	91.8	95.2	66.2	79.6	91.9	92.3	93.6	95.3	95.4	94.6	95.9	96.4
	10	65.3	78.0	89.4	92.1	90.5	91.5	92.8	93.3	94.3	92.1	68.5	80.1	93.8	93.7	95.7	95.4	94.9	95.4	96.7	96.7
	15	63.6	79.8	91.0	91.4	90.4	94.8	88.4	**91.4**	91.1	90.1	68.3	81.4	93.6	95.8	96.1	97.1	97.5	**97.2**	97.7	97.7
	20	64.3	77.8	91.2	88.0	90.7	91.3	88.4	94.8	95.8	97.5	70.3	82.1	94.5	95.5	96.7	96.5	96.8	96.8	97.5	97.5
	25	62.0	78.5	82.7	92.8	91.0	95.2	93.1	93.6	91.6	88.3	66.2	73.4	91.8	92.5	93.7	96.1	97.0	95.9	97.0	95.9
PCA	5	73.0	82.9	91.2	95.7	96.9	97.5	97.6	97.9	97.8	97.8	73.1	81.3	89.9	92.2	95.0	94.5	95.8	97.0	97.0	97.2
	10	75.8	84.9	93.5	96.9	97.6	98.0	98.2	98.3	98.4	98.4	73.8	84.2	92.6	95.0	96.0	90.7	96.9	96.8	97.8	97.8
	15	77.0	88.4	94.5	97.5	97.9	98.5	98.4	**98.5**	98.6	98.4	76.4	86.8	93.6	96.4	97.1	97.1	97.4	**97.5**	97.3	97.4
	20	78.4	89.4	95.2	97.9	98.1	98.6	98.6	98.7	98.8	98.7	77.5	87.0	93.8	96.6	96.5	97.0	96.8	96.6	97.0	96.8
	25	49.3	71.0	95.3	98.2	98.3	98.8	99.0	98.4	98.7	98.5	73.7	78.0	88.8	93.9	94.8	96.2	96.1	96.6	96.4	96.8
PCA with rbf kernel	5	72.1	85.7	91.2	95.4	96.5	97.0	96.9	97.1	97.5	97.6	68.9	81.8	91.4	94.0	96.9	97.0	96.7	97.2	96.9	97.6
	10	73.8	88.3	95.2	97.2	97.6	97.9	97.9	98.2	98.1	98.1	74.6	85.7	93.8	95.2	97.1	97.5	97.4	98.0	98.0	98.3
	15	74.2	91.1	95.9	97.1	97.6	98.2	98.3	**98.5**	98.3	98.3	77.0	87.2	94.9	97.0	97.5	97.6	98.0	**98.1**	97.8	98.0
	20	75.0	73.6	96.5	97.6	98.0	97.7	98.3	98.6	98.6	98.3	73.9	89.4	95.6	97.2	97.7	97.6	97.7	97.6	97.8	97.9
	25	73.3	91.4	97.1	97.9	98.3	98.0	98.8	98.4	98.6	98.5	71.3	76.4	86.8	91.9	96.2	97.4	96.4	96.7	96.6	95.7

Fig. 9. Overall accuracy results for TDNN and LSTM algorithms and for NMF, PCA and kPCA dimensionality reduction methods. The rows display the size of time window in samples and the columns the number of components used.

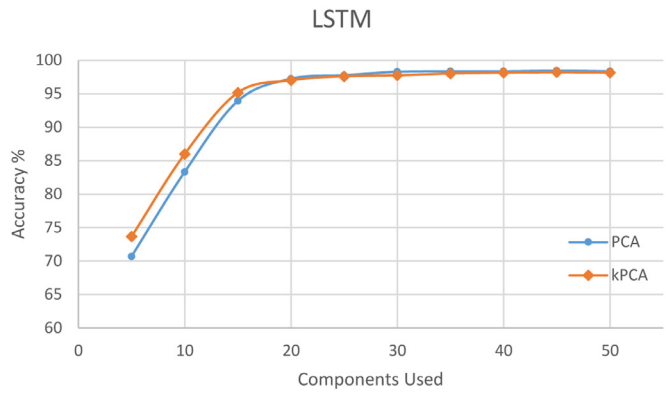

Fig. 10. Average accuracy per number of components for all time windows for LSTM classification algorithm.

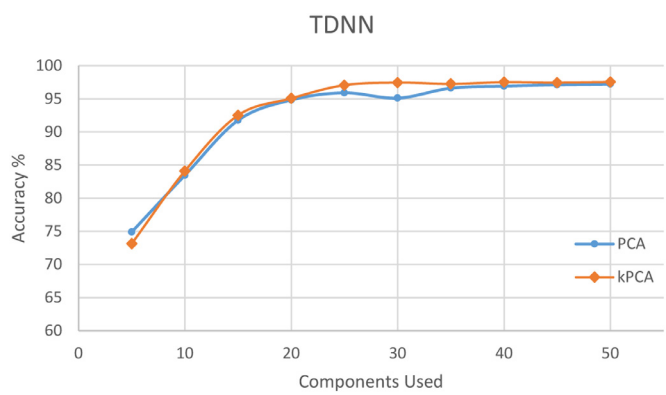

Fig. 11. Average accuracy per number of components for all time windows for TDNN classification algorithm.

Table 4
Accuracy for TDNN and LSTM algorithms using NMF, PCA and kPCA decomposition method for sequence length = 15 and 40 components.

	TDNN			LSTM		
Fault	NMF	PCA	kPCA	NMF	PCA	kPCA
Normal	89.9	93.0	98.0	29.1	99.8	99.4
1	99.9	99.9	99.7	99.8	100.0	100.0
2	99.6	99.3	99.3	99.2	99.7	99.8
3	91.7	94.9	99.1	100.0	100.0	100.0
4	99.6	99.5	99.6	99.7	100.0	100.0
5	99.7	99.4	99.6	100.0	100.0	100.0
6	100.0	100.0	100.0	100.0	100.0	100.0
7	100.0	100.0	100.0	100.0	100.0	100.0
8	98.8	98.4	98.8	98.7	99.1	99.1
9	81.5	84.2	85.0	24.1	84.8	85.6
10	98.4	97.9	97.8	99.0	98.6	97.6
11	99.6	99.6	99.6	100.0	100.0	100.0
12	99.3	98.7	99.2	98.4	99.7	99.6
13	97.0	97.1	97.1	96.8	97.7	97.4
14	99.9	99.8	99.9	99.9	99.9	100.0
17	97.7	97.8	97.9	98.2	98.3	98.3
18	97.5	96.3	96.3	98.4	97.2	98.0
19	99.6	99.4	99.5	99.8	99.9	99.9
20	96.5	96.7	96.9	96.3	97.2	97.0
average	**97.2**	**97.5**	**98.1**	**91.4**	**98.5**	**98.5**

times more frequent sampling rate leads to earlier fault detection. However, this study aims to provide an efficient solution that takes into account both time and accuracy for the fault detection task. For a balanced evaluation of the models generated, a combination of 15 samples for the window size and 40 components is used. The results are presented in Table 4

Table 5
Benchmarking table. [1]Zhang and Zhao (2017), [2]Wu and Zhao (2018), [3]Zhang et al. (2019),[4]this work. For the calculation of the average accuracy, results from Faults 15 and 16 and normal operation if not provided are excluded.

	DBN[1]	DCNN[2]	BiLSTM[3]	LSTM PCA[4]	LSTM kPCA[4]
Normal	–	97.8	97.7	99.8	99.4
1	100.0	98.6	98.9	100.0	100.0
2	99.0	98.5	96.1	99.7	99.8
3	95.0	91.7	96.1	100.0	100.0
4	98.0	97.6	94.2	100.0	100.0
5	86.0	91.5	99.6	100.0	100.0
6	100.0	97.5	100	100.0	100.0
7	100.0	99.9	99.8	100.0	100.0
8	78.0	92.2	84.1	99.1	99.1
9	57.0	58.4	69.9	84.8	85.6
10	98.0	96.4	98.4	98.6	97.6
11	87.0	98.4	96.3	100.0	100.0
12	85.0	95.6	98.8	99.7	99.6
13	88.0	95.7	85.8	97.7	97.4
14	87.0	98.7	100	99.9	100.0
17	100.0	94.5	96.8	98.3	98.3
18	98.0	93.9	89.8	97.2	98.0
19	93.0	98.5	91.6	99.9	99.9
20	93.0	93.3	97.5	97.2	97.0
average	91.22	94.14	94.28	**98.5**	**98.5**
Sampl. time	3 min	3 min	2 min	**36 s**	**36 s**

Compared to similar studies, our work achieves better results in both accuracy and time as depicted in Table 5. We have to note, that those results are obtained because of the preprocessing procedure which is applied in this work, combined with the exclusion of Faults 15 and 16 from the classification models as explained in detail in Section 3.4.

7. Conclusion

In this work, an FDD framework that relies on a Dynamic Neural Network is developed which can be easily adapted in many dynamic non-linear processes. We present a new reference dataset for the Tennessee Eastman Process that is based on the most recent simulator. The effectiveness of the FDD framework was evaluated with two state-of-the-art Time Series classification methods namely the LSTM and the TDNN. Moreover, several feature reduction methods (MI, mRMR, NMF, PCA and kPCA) were applied. The results demonstrate the importance of selecting a suitable feature reduction method and the impact of the size of the time window that can help in getting high accuracy for the detection of faults. The experimental results demonstrate that the detection performance of the proposed FDD is enhanced, whereas the comparative analysis proves that the proposed approach has better performance in fault detection. Moreover, it is highlighted that our method has decreased fault detection time than other studies, since the sampling rate of the data was increased up to five times. Finally, based on the aforementioned analysis, the FDD framework is considered highly efficient and has the ability to identify faults based on the proposed multiclass approach.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Akbaryan, F., Bishnoi, P.R., 2001. Fault diagnosis of multivariate systems using pattern recognition and multisensor data analysis technique. Comput. Chem. Eng. 25 (9), 1313–1339. doi:10.1016/S0098-1354(01)00701-3.

Angelopoulos, A., Michailidis, E.T., Nomikos, N., Trakadas, P., Hatziefremidis, A., Voliotis, S., Zahariadis, T., 2020. Tackling faults in the industry 4.0 era—A survey of machine-learning solutions and key aspects. Sensors 20 (1). doi:10.3390/s20010109.

Bathelt, A., Ricker, N.L., Jelali, M., 2015. Revision of the Tennessee Eastman process model. IFAC-PapersOnLine 48 (8), 309–314. doi:10.1016/j.ifacol.2015.08.199. 9th IFAC Symposium on Advanced Control of Chemical Processes ADCHEM 2015

Chen, J., Liao, C.-M., 2002. Dynamic process fault monitoring based on neural network and PCA. J. Process Control 12 (2), 277–289. doi:10.1016/S0959-1524(01)00027-0.

Chiang, L.H., Kotanchek, M.E., Kordon, A.K., 2004. Fault diagnosis based on Fisher discriminant analysis and support vector machines. Comput. Chem. Eng. 28 (8), 1389–1401. doi:10.1016/j.compchemeng.2003.10.002.

Christofides, P.D., Davis, J.F., El-Farra, N.H., Clark, D., Harris, K.R., Gipson, J.N., 2007. Smart plant operations: vision, progress and challenges. AIChE J. 53 (11), 2734–2741.

Downs, J.J., Vogel, E.F., 1993. A plant-wide industrial process control problem. Comput. Chem. Eng. 17 (3), 245–255. doi:10.1016/0098-1354(93)80018-I.

Eslamloueyan, R., 2011. Designing a hierarchical neural network based on fuzzy clustering for fault diagnosis of the Tennessee–Eastman process. Appl. Soft Comput. 11 (1), 1407–1415.

Ghosh, K., Ramteke, M., Srinivasan, R., 2014. Optimal variable selection for effective statistical process monitoring. Comput. Chem. Eng. 60, 260–276. doi:10.1016/j.compchemeng.2013.09.014.

Heo, S., Lee, J.H., 2018. Fault detection and classification using artificial neural networks. IFAC-PapersOnLine 51 (18), 470–475. doi:10.1016/j.ifacol.2018.09.380.

Hochreiter, S., Schmidhuber, J., 1997. Long short-term memory. Neural Comput. 9 (8), 1735–1780. doi:10.1162/neco.1997.9.8.1735.

Iqbal, R., Maniak, T., Doctor, F., Karyotis, C., 2019. Fault detection and isolation in industrial processes using deep learning approaches. IEEE Trans. Ind. Inf. 15 (5), 3077–3084.

Kano, M., Nagao, K., Hasebe, S., Hashimoto, I., Ohno, H., Strauss, R., Bakshi, B.R., 2002. Comparison of multivariate statistical process monitoring methods with applications to the eastman challenge problem. Comput. Chem. Eng. 26 (2), 161–174.

Larsson, T., Hestetun, K., Hovland, E., Skogestad, S., 2001. Self-optimizing control of a large-scale plant: the Tennessee Eastman process. Ind. Eng. Chem. Res. 40 (22), 4889–4901. doi:10.1021/ie000586y.

Lau, C., Ghosh, K., Hussain, M., Hassan, C.C., 2013. Fault diagnosis of Tennessee Eastman process with multi-scale PCA and ANFIS. Chemom. Intell. Lab. Syst. 120, 1–14.

Lee, D.D., Seung, H.S., 1999. Learning the parts of objects by non-negative matrix factorization. Nature 401 (6755), 788–791.

Lin, W., Qian, Y., Li, X., 2000. Nonlinear dynamic principal component analysis for on-line process monitoring and diagnosis. Comput. Chem. Eng. 24 (2–7), 423–429.

Maurya, M.R., Rengaswamy, R., Venkatasubramanian, V., 2005. Fault diagnosis by qualitative trend analysis of the principal components. Chem. Eng. Res. Des. 83 (9), 1122–1132.

Odiowei, P., Cao, Y., 2010. State-space independent component analysis for nonlinear dynamic process monitoring. Chemom. Intell. Lab. Syst. 103 (1), 59–65.

Peng, H., Long, F., Ding, C., 2005. Feature selection based on mutual information criteria of max-dependency, max-relevance, and min-redundancy. IEEE Trans. Pattern Anal. Mach. Intell. 27 (8), 1226–1238.

Pistikopoulos, E., Barbosa-Povoa, A., Lee, J.H., Misener, R., Mitsos, A., Reklaitis, G., Venkatasubramanian, V., You, F., Gani, R., 2021. Process systems engineering–the generation next? Comput. Chem. Eng. 107252.

Qin, J., Liu, Y., Grosvenor, R., 2016. A categorical framework of manufacturing for industry 4.0 and beyond. Procedia CIRP 52, 173–178. doi:10.1016/j.procir.2016.08.005. The Sixth International Conference on Changeable, Agile, Reconfigurable and Virtual Production (CARV2016)

Rad, M.A.A., Yazdanpanah, M.J., 2015. Designing supervised local neural network classifiers based on em clustering for fault diagnosis of Tennessee Eastman process. Chemom. Intell. Lab. Syst. 146, 149–157.

Ricker, N.L., 1996. Decentralized control of the Tennessee Eastmanchallenge process. J. Process Control 6 (4), 205–221.

Russell, E.L., Chiang, L.H., Braatz, R.D., 2000. Fault detection in industrial processes using canonical variate analysis and dynamic principal component analysis. Chemom. Intell. Lab. Syst. 51 (1), 81–93. doi:10.1016/S0169-7439(00)00058-7.

Sansana, J., Joswiak, M.N., Castillo, I., Wang, Z., Rendall, R., Chiang, L.H., Reis, M.S., 2021. Recent trends on hybrid modeling for industry 4.0. Comput. Chem. Eng. 107365.

Scholkopf, B., Smola, A., Müller, K.-R., 1999. Kernel principal component analysis. In: Advances in Kernel Methods - Support Vector Learning. MIT Press, pp. 327–352.

Venkatasubramanian, V., Rengaswamy, R., Yin, K., Kavuri, S.N., 2003. A review of process fault detection and diagnosis: part I: quantitative model-based methods. Comput. Chem. Eng. 27 (3), 293–311.

Wu, H., Zhao, J., 2018. Deep convolutional neural network model based chemical process fault diagnosis. Comput. Chem. Eng. 115, 185–197.

Xie, Z., Yang, X., Li, A., Ji, Z., 2019. Fault diagnosis in industrial chemical processes using optimal probabilistic neural network. Can. J. Chem. Eng. 97 (9), 2453–2464.

Yin, S., Ding, S.X., Haghani, A., Hao, H., Zhang, P., 2012. A comparison study of basic data-driven fault diagnosis and process monitoring methods on the benchmark Tennessee Eastman process. J. Process Control 22 (9), 1567–1581. doi:10.1016/j.jprocont.2012.06.009.

Zhang, S., Bi, K., Qiu, T., 2019. Bidirectional recurrent neural network-based chemical process fault diagnosis. Ind. Eng. Chem. Res. 59 (2), 824–834.

Zhang, Y., 2009. Enhanced statistical analysis of nonlinear processes using KPCA, KICA and SVM. Chem. Eng. Sci. 64 (5), 801–811. doi:10.1016/j.ces.2008.10.012.

Zhang, Z., Zhao, J., 2017. A deep belief network based fault diagnosis model for complex chemical processes. Comput. Chem. Eng. 107, 395–407.

Zhao, H., Lai, Z., Chen, Y., 2019. Global-and-local-structure-based neural network for fault detection. Neural Netw. 118, 43–53.

Zhao, H., Sun, S., Jin, B., 2018. Sequential fault diagnosis based on LSTM neural network. IEEE Access 6, 12929–12939.